Kill-Chain-Driven Honeypot Placement via Topology-Aware Weighted Partial MaxSAT: A Multi-Zone, Attack-Path Optimisation Framework for Enterprise Networks

Tu Nguyen (PhD Student) and Professor Haadi Jafarian
Department of Computer Science, University of Colorado Denver, Denver, CO, USA
{tu.nguyen, haadi.jafarian}@ucdenver.edu

***Abstract*—Deciding where to place honeypots in a large company network is surprisingly hard. The network is split into different security zones, contains thousands of different types of servers, and faces attackers who follow well-known step-by-step attack patterns called kill chains. On top of that, the organisation has a limited budget, and some honeypot types cannot be deployed at the same time because they would conflict with each other.**

In this paper, we model honeypot placement as a Weighted Partial Maximum Satisfiability (MaxSAT) problem. We encode six goals at once: how well we detect threats, how many attack techniques we cover from the MITRE ATT&CK framework, how many attack tactic families we cover, how well we stop attackers early, how well we catch attackers at the end of their path, and how good our coverage is across all attack paths. All hard rules—budget limits, zone isolation, and conflict constraints—are encoded as clauses that must never be violated. We formally prove that the honeypot placement decision problem is NP-hard, justifying why a complete solver like RC2 is necessary over any greedy heuristic.

We test our approach on networks ranging from 100 nodes (tiny) to over 5.5 million nodes (very large), simulating five different real-world attack scenarios. Our MaxSAT solver achieves a quality score of $Q = 67.4$ compared to $Q = 52.8$ for the best competing approach ($+27.8\%$), while also providing a mathematical guarantee that its answer is the best possible solution.

***Index Terms*—Honeypot placement, Weighted Partial MaxSAT, kill chain, MITRE ATT&CK, network deception, multi-zone topology, NP-hardness, proactive cyber defence, RC2 solver, enterprise network security.**

I. Introduction

A HONEYPOT is a fake computer system designed to look attractive to attackers. When an attacker interacts with it, the defender learns about their methods and can raise an alert. Honeypots are widely used in network security, but their usefulness depends heavily on *where* they are placed. If a honeypot is put in the wrong place, it will never be visited by attackers and the investment is wasted.

Real company networks make this placement decision much harder than it sounds. Consider a typical large organisation. It has an internet-facing area (called a DMZ) where its public web servers sit. It has an internal office network. It might have cloud services. It may also have industrial control systems (called OT/SCADA) that run physical equipment like power plants or factories. Each of these areas has different security rules, different types of computers, and different risks. Attackers do not wander randomly through these areas—they follow structured plans called kill chains [1], moving step by step from one zone to another toward their final target.

The problem is further complicated by three facts. First, the organisation has a limited budget and cannot afford to deploy a honeypot on every machine. Second, certain combinations of honeypot types are incompatible—for example, an industrial control system honeypot deployed in the cloud zone would be useless because the OT zone is physically isolated from the cloud. Third, there are many possible attack paths through the network and an ideal placement should intercept as many as possible.

Existing methods do not handle all these challenges at the same time. Simple rule-based approaches just follow fixed guidelines. Greedy algorithms pick the best-looking option at each step but cannot reverse a bad decision. Game-theoretic approaches [8], [11] model honeypot placement as a strategic interaction but assume flat networks without zone isolation or budget conflict constraints. Reinforcement learning methods [12] learn how to *engage* attackers after placement decisions have already been made, but do not optimise the initial placement itself. In this paper, we use a formal solver that considers *all* constraints and objectives together and provides a *proven optimal* answer. We also formally prove that the problem is NP-hard (Theorem 1), which means that no polynomial-time algorithm can always find the optimal placement—making our complete MAXSAT solver not just a good choice, but the correct one.

Our main contributions are:

1) **A six-goal placement objective** encoded as a single MaxSAT optimisation problem comprising: (1) detection efficiency, (2) coverage of MITRE ATT&CK attack techniques, (3) coverage of MITRE ATT&CK tactic families, (4) catching attackers early in their path, (5) catching attackers at the end of their path for forensic evidence, and (6) covering as many attack paths as possible. Each of these six objectives addresses a different failure mode of prior methods, and together they create a solver that is simultaneously prevention-oriented, intelligence-driven, budget-aware, and topology-correct.

2) **A formal NP-hardness proof** showing that the honeypot placement decision problem is NP-hard via polynomial-time reduction from Weighted Maximum Coverage. This result formally justifies why a complete solver is necessary over any greedy or heuristic approach.
3) **A cross-path force-multiplier encoding** that lets the solver discover—and exploit—situations where one honeypot simultaneously satisfies soft clauses across multiple attack paths, giving a $2.1\times$ advantage over greedy methods that evaluate only one path at a time.
4) **A thorough experimental evaluation** across six different network sizes, using a Monte Carlo simulation to model attacker behaviour and testing what happens when threat intelligence updates cause us to re-run the solver with new priorities.

The rest of this paper is organised as follows. Section II discusses related work. Section III defines the problem formally and proves NP-hardness. Section IV presents our MaxSAT model. Section V describes our experiments. Section VI presents the results. Section VII explains why our approach beats the baselines. Section VIII covers limitations and future work. Section IX concludes.

II. Related Work

This section reviews five bodies of research most relevant to our work: honeypot placement, game-theoretic and reinforcement-learning-based deception, MITRE ATT&CK-based defence, formal methods for network security, and moving-target defence. For each body of work we describe what it achieves, explain its limitations relative to our problem, and identify the specific gap it leaves. We then provide a systematic novelty analysis with a capability matrix (Figure 1) and an end-to-end pros-and-cons comparison table (Table I) that maps every baseline against our approach across all six objectives.

A. Honeypot Placement Research

Honeypots have been studied for decades. Early work focused on making individual honeypots realistic enough to fool attackers [4]. Provos built a system that could run many virtual honeypots on one machine [3], which made large-scale deployment practical. Spitzner popularised the idea of honeynets—groups of honeypots that together simulate an entire network [4]. However, most of this early work assumed a human expert would decide where to put the honeypots; there was no automated decision-making process.

Pros: These works established the foundational concepts of honeypot realism and honeynet coordination. **Cons:** No automated placement, no budget constraints, no multi-zone topology, and no kill-chain awareness. They cannot be directly used in a large enterprise network without substantial manual engineering effort.

B. Game-Theoretic Honeypot Placement

Game theory has been the dominant framework for automated honeypot placement decisions over the past decade. Kiekintveld *et al.* [8] provided one of the earliest rigorous game-theoretic analyses of honeypot deployment, modelling the interaction as a Stackelberg game where the defender commits to a placement strategy and the attacker responds optimally. They showed that this formulation leads to mixed-strategy equilibria that are strictly better than any deterministic placement. Yuill *et al.* [9] were among the first to frame deceptive file placement as a strategic interaction. Tan *et al.* [10] and Pawlick *et al.* [11] extended this line of work using formal game theory to model how rational attackers respond to deception, providing a comprehensive taxonomy of defensive deception strategies across six categories including honeypots, obfuscation, and moving-target techniques. Wang *et al.* [13] surveyed this entire landscape, reviewing over 30 papers on cyber deception and identifying the key open problems that remain unsolved.

Pros: Game-theoretic methods are mathematically rigorous. They model rational attacker behaviour explicitly, produce equilibrium strategies that are provably stable, and can handle uncertainty in attacker knowledge. Kiekintveld *et al.* [8] in particular provide a strong theoretical foundation for why honeypot placement must be randomised in adversarial settings.

Cons: All of these approaches assume a flat, homogeneous network where every node is equivalent. They cannot encode physical air gaps between security zones (D1), do not model kill-chain step ordering or early-interception bonuses (D2), and do not integrate ATT&CK objectives (D3). The solution concept they produce—a Nash or Stackelberg equilibrium—is also fundamentally different from a proven weighted optimum: an equilibrium is a stable strategy pair, not the best possible placement under a multi-objective scoring function. Furthermore, none of these methods can enforce pairwise honeypot conflict constraints (D4), which are essential in environments where deploying certain honeypot types together is physically impossible (*e.g.* SCADA honeypots cannot coexist with DMZ honeypots when those zones are air-gapped).

Gap: Game-theoretic methods fail on D1, D2, D3, and D4 simultaneously. Our work is complementary: we solve the *placement* optimisation problem, and a game-theoretic engagement strategy [12] can be layered on top.

C. Reinforcement Learning for Honeypot Engagement

More recently, reinforcement learning (RL) has been applied to the honeypot problem. Huang and Zhu [12] modelled the honeypot interaction as a Semi-Markov Decision Process and used RL to learn an adaptive engagement policy that decides how long to keep an attacker engaged with a honeypot before alerting. Their work showed that RL-based engagement significantly outperforms fixed-threshold policies in terms of intelligence gathering.

Pros: RL-based engagement learns from experience and adapts to attacker behaviour over time. The Semi-Markov formulation captures the time-varying nature of attacker interactions, which static models cannot. Their approach is complementary to placement optimisation.

Cons: The Huang and Zhu [12] framework assumes that honeypots are already deployed at fixed locations. It optimises

the *engagement policy* given a placement, not the *placement* itself. It also assumes a flat network with no zone structure (D1), no kill-chain ordering (D2), no ATT&CK integration (D3), no budget or conflict constraints (D4), and no optimality certificate (D5). Training RL policies also requires many episodes of attacker-honeypot interaction, which is difficult to obtain without a live deployment.

Gap: RL engagement and MAXSAT placement are complementary, not competing. Our solver decides *where* to put honeypots; RL decides *how* to interact with attackers once they arrive. A natural extension of our work would combine our optimal placement with an RL engagement layer.

D. Using MITRE ATT&CK for Defence Planning

MITRE ATT&CK [2] is a publicly available catalogue of the tactics and techniques that real attackers use in practice. It has become the standard reference for both offensive and defensive security research.

Applebaum *et al.* [14] were among the first to use ATT&CK for automated security testing. They built a framework that acts like an automated red team, moving through a network and testing defences using techniques from ATT&CK.

Pros: ATT&CK-based red-teaming produces realistic attack simulations that mirror real adversary behaviour. The framework covers D2 (kill-chain-structured paths) and D3 (ATT&CK techniques), and it has been validated against real-world threat intelligence.

Cons: It is a red-team tool, not a placement optimiser. Its goal is to test *existing* defences, not to decide where to place *new* ones. It provides no budget enforcement (D4) and no optimality guarantee (D5).

Hemberg *et al.* [15] used evolutionary (genetic) algorithms to select the best defensive controls from ATT&CK. **Pros:** GA-based methods achieve partial ATT&CK coverage and can explore large solution spaces. **Cons:** They operate on a flat control catalogue with no zone structure (D1), no kill-chain ordering (D2), no budget or conflict constraints (D4), and no optimality guarantee (D5). Critically, genetic algorithms evaluate fitness one objective at a time. This means they cannot discover situations where deploying one honeypot simultaneously satisfies clauses for multiple attack paths—the cross-path force-multiplier effect that gives our solver a $2.1\times$ advantage on technique coverage (see Section II-G4).

Gap: Both works use ATT&CK to evaluate or select defences, but neither embeds ATT&CK technique weights, stealth scores, or tactic-family bonuses directly into a solver as live optimisation objectives (D3). Our work is the first to do this.

E. Using Formal Methods for Network Security

Formal methods use mathematical logic to prove properties of systems. In network security, they have been used to check firewall rules for consistency [16], automatically generate network configurations [17], and verify security policies [18].

Pinisetty *et al.* [19] applied MAXSAT to the problem of selecting the best rules for an intrusion detection system. **Pros:** Their work shows that MAXSAT is a practical tool for security resource allocation and that the RC2 solver can handle realistic security problem sizes. **Cons:** IDS rule selection and honeypot placement are fundamentally different problems. Their model has no zone structure (D1), no kill-chain ordering (D2), no ATT&CK coverage objective (D3), and the optimality guarantee is partial because they use approximation.

Saha and Gera [20] used ILP to find the optimal locations for security sensors in enterprise networks. **Pros:** ILP achieves multi-zone support (D1), budget constraint enforcement (D4), and proven optimality (D5). This is the closest technical predecessor to our work. **Cons:** Their model does not encode kill-chain step ordering or early-interception priority (D2), does not integrate ATT&CK technique weights or tactic-family bonuses (D3), and faces a serious scalability problem: ILP becomes very slow as the problem size grows and cannot handle the 5.5-million-node networks we evaluate within a 60-second time limit. Our zone-decomposed MAXSAT approach solves the XLarge network (500,000 nodes) in 30 seconds by splitting the instance into five parallel zone sub-problems.

Jajodia *et al.* [22] introduced topological analysis of network attack vulnerability. **Pros:** Their chokepoint insight is foundational. They showed that certain network edges carry all attacker traffic, making them the most valuable places to defend. **Cons:** They provide no automated placement algorithm, no ATT&CK integration (D3), no budget enforcement (D4), and no optimality guarantee (D5). Our work operationalises their insight: chokepoints appear in our model as high path-weight edges, causing the solver to concentrate honeypots there naturally.

Gap: No prior work has applied Weighted Partial MAXSAT to honeypot placement with kill-chain-aware objectives and physical zone isolation constraints.

F. Moving-Target Defence

Moving-target defence (MTD) [23] is a strategy where the defender constantly changes the network configuration to confuse attackers. MTD and honeypot placement are complementary ideas rather than competing ones: MTD changes the *real* system to create uncertainty, while honeypots add *fake* systems to create traps.

Pros: MTD provides dynamic defence that prevents attackers from building a stable picture of the network. Zone-aware MTD implementations respect network topology (D1) and can respond to new threats in real time.

Cons: MTD work does not address the honeypot placement subproblem, kill-chain ordering (D2), ATT&CK integration (D3), conflict constraints (D4), or optimality guarantees (D5). Our adaptive redeployment algorithm (Algorithm 1) adds a dynamic element that bridges toward MTD philosophy: when new threat intelligence changes attack path probabilities, our solver re-optimises using the previous solution as a warm start.

G. Novelty Analysis: Related Work vs. This Paper

We now demonstrate systematically that our work is novel. We map each body of related work to five capability dimensions (D1–D5), show that no existing method satisfies all five simultaneously, and then provide a full end-to-end pros-and-cons comparison across all six solver objectives.

Approach	D1 Multi-zone	D2 Kill-chain	D3 ATT&CK	D4 Budget +Conflict	D5 Proven Optimal
Kiekintveld [8]	✗	✗	✗	~	Stack.
Pawlick *et al.* [11]	✗	✗	✗	~	Nash
RL engagement [12]	✗	✗	✗	✗	✗
GA-based [15]	✗	✗	✓	✗	✗
ILP sensor [20]	✓	✗	✗	✓	✓
ATT&CK emulation [14]	~	✓	✓	✗	✗
MTD [23]	✓	✗	✗	✗	✗
Topology analysis [22]	✓	~	✗	✗	✗
Greedy-BiDir (our baseline)	✓	✓	~	✗	✗
Ours (MaxSAT-RC2)	✓	✓	✓	✓	**Proven**

✓ Fully satisfied ~ Partial ✗ Not addressed

Fig. 1: Novelty gap: related work vs. this paper across five capability dimensions D1–D5. No prior method satisfies all five dimensions simultaneously. Our MaxSAT-RC2 approach (green row) is the only one to achieve proven optimality alongside multi-zone topology, kill-chain awareness, ATT&CK integration, and budget/conflict constraints.

1) The Five Capability Dimensions:

D1 Multi-zone topology with air gaps. The method must respect physical isolation between zones.

D2 Kill-chain-aware placement. Attack paths are ordered sequences of steps; the method must prioritise early interception over forensic detection.

D3 MITRE ATT&CK integration. ATT&CK technique weights, stealth scores, and tactic-family coverage must be live objectives inside the solver.

D4 Budget and conflict constraints. The method must enforce a global budget, per-zone budgets, and pairwise conflict rules as hard constraints.

D5 Proven optimality. The method must provide a mathematical certificate that no better placement exists within the constraints.

2) End-to-End Pros and Cons vs. Our Six Objectives: Table I maps every related work against our six solver objectives (O1–O6), showing which objectives each method handles well, which it handles partially, and which it cannot address at all. This table directly answers the question: *which specific capability is missing from each prior work, and which objective in our MAXSAT formulation fills that gap?*

3) The Six Objectives: Why Each One Is Necessary: Each of the six objectives in our MAXSAT formulation was chosen because removing it would leave a gap that an attacker could exploit. We explain why each objective is necessary, what prior work misses by omitting it, and how our solver encodes it.

O1 — Detection Efficiency (Level-1 soft clauses, weight $W_{j,a}$). This is the most basic objective: we want each honeypot to detect as many attacker actions as possible per dollar spent. Without this objective, a solver might deploy honeypots in zones where no attack is likely. Prior work including ILP sensor placement [20] handles this objective but does not combine it with the other five. We encode it as Level-1 soft clauses, giving the solver credit for every (t_j, a) pair it covers even if those pairs are not on any specific kill-chain path.

O2 — ATT&CK Technique Coverage (Level-2 per-technique clauses, weight $\sum_a W_{j,a}$). Covering many different ATT&CK techniques ensures that our honeypots can detect diverse attacker behaviours, not just one type of attack. GA-based methods [15] partially address this but without budget constraints or zone topology. Our solver encodes it as Level-2 per-technique clauses: for each ATT&CK technique t_j, the solver is rewarded for detecting it at least once anywhere in the network. This pushes the solver toward a portfolio of honeypot types that together cover many different columns of the ATT&CK matrix.

O3 — Tactic-Family Breadth (Level-2 family bonus, weight $1.2 \times \sum_{j \in f} \sum_a W_{j,a}$). Coverage of 10 techniques in one tactic family (say, Lateral Movement) is much less valuable than coverage of one technique in each of 10 different tactic families. An attacker who is only detected during Lateral Movement but not during Initial Access, Execution, or Exfiltration can simply shift their tactics. No prior method includes tactic-family breadth as a live objective. We encode it as a $1.2\times$ bonus soft clause: the solver earns an extra 20% credit for being the first honeypot to cover any technique in a new tactic family. This is the reason we achieve FamCov $= 100\%$ at Medium scale.

O4 — Early Interception (Level-4 soft clauses, weight $\rho_\pi \times \max_{h<|\pi|}(iv_{\pi,h}) \times \sum_{j,a} W_{j,a}$). Catching an attacker at the final step of their path (after data exfiltration) is fundamentally less valuable than catching them at the first step (before any damage occurs). Game-theoretic methods [8], [11] and RL engagement methods [12] do not encode this preference for prevention over forensics. ATT&CK emulation [14] models kill-chain steps but is a red-team tool, not a placement optimiser. We encode early interception as Level-4 soft clauses—the highest priority level—where the solver earns a large bonus for catching the attacker at any non-final hop. Combined with hard constraint C7 (which requires that the $e_\pi = 1$ variable can only be set if a non-final hop is covered), this makes the solver strongly prefer preventive placements over forensic ones.

O5 — Forensic Backward Coverage (Level-3 backward clauses, weight $\rho_\pi \cdot iv_{\pi,h} \cdot W_{j,a} \times 0.7$). Even when we cannot catch an attacker early, covering the later steps of their path still provides forensic value: we learn what techniques they used and can strengthen defences for the future. We encode backward coverage as Level-3 soft clauses with a 0.7 discount relative to forward coverage. This 0.7 factor is the formal encoding of the distinction between prevention and forensics that greedy baselines treat as equivalent.

O6 — Multi-Path Coverage (Level-3 forward clauses across all paths simultaneously). Most prior methods evaluate one attack path at a time. Our solver encodes *all* five attack paths as clauses simultaneously. This allows the solver

TABLE I: End-to-End Pros and Cons of Related Work Mapped to Our Six Solver Objectives. O1=Detection efficiency, O2=ATT&CK technique coverage, O3=Tactic-family coverage, O4=Early interception, O5=Forensic backward coverage, O6=Multi-path coverage.

Method	Key strength	Key weakness vs. our work	O1 DetEff	O2 TechCov	O3 FamCov	O4 Early	O5 Forensic	O6 MultiPath
Kiekintveld [8]	Stackelberg equilibrium; rational attacker model	Flat network; no ATT&CK; no zone or conflict constraints	~	✗	✗	✗	✗	✗
Pawlick *et al.* [11]	Comprehensive deception taxonomy; Nash equilibrium	No kill-chain ordering; no multi-zone; no budget constraints	~	✗	✗	✗	✗	✗
Huang & Zhu [12]	Adaptive RL engagement; time-aware attacker model	Optimises engagement not placement; no zone or ATT&CK	✗	✗	✗	✗	✗	✗
Wang *et al.* [13]	Comprehensive cyber deception survey; identifies open problems	Survey only; no placement algorithm or solver	✗	✗	✗	✗	✗	✗
GA-based [15]	Partial ATT&CK technique selection; large search space	No zone topology; no budget; no kill-chain; no proof	~	~	✗	✗	✗	✗
ILP sensor [20]	Proven optimal; multi-zone; budget-aware	No kill-chain; no ATT&CK; does not scale past 50k nodes	✓	✗	✗	✗	✗	✗
ATT&CK emulation [14]	Kill-chain paths; ATT&CK techniques; realistic simulation	Red-team tool; no placement optimisation; no budget	~	✓	~	✓	~	~
MTD [23]	Dynamic defence; zone-aware routing	Moves real assets; no honeypot placement; no ATT&CK	✗	✗	✗	✗	✗	✗
Topology [22]	Chokepoint analysis; attack-graph model	No placement algorithm; no ATT&CK; no solver	~	✗	✗	✗	✗	~
Greedy-BiDir	Multi-zone; kill-chain; bidirectional coverage	No conflict constraints; no proof; one path at a time	✓	~	✗	~	~	~
Ours (MaxSAT-RC2)	**All six objectives; proven optimal; scalable**	**None of the above gaps**	✓	✓	✓	✓	✓	✓

✓ = fully addresses objective; ~ = partially addresses; ✗ = does not address.

to discover the cross-path force-multiplier effect described in Section II-G4: a single honeypot that covers technique T1021 in zone2 satisfies Level-3 clauses for three different attack paths at once, earning 3.612 total weight versus greedy's 1.749 for the same deployment decision.

4) The Cross-Path Force-Multiplier: Deep Analysis: The most important reason why our method outperforms all baselines is not just its optimality certificate—it is a fundamental difference in how deployment decisions are evaluated. We call this the **cross-path force-multiplier effect**, and we now explain it in detail.

Every attack path in our model is encoded as a set of Level-3 soft clauses, one per (path, hop) pair. The key insight is that the same ATT&CK technique appearing in the same network zone across multiple attack paths creates multiple separate soft clauses—all of which are satisfied simultaneously when a single honeypot is deployed in that zone.

What the force-multiplier means geometrically. Think of the solver's decision space as a high-dimensional space where each dimension corresponds to one soft clause. A greedy method moves through this space one dimension at a time—it finds the honeypot that satisfies the most valuable single clause and deploys it. Our solver, by contrast, searches this space globally and finds deployments that satisfy *many clauses at once*, even when those clauses belong to completely different attack paths. The force-multiplier is the ratio between the total clause weight satisfied by a globally-optimal deployment and the weight a greedy method would count for the same deployment.

The force-multiplier works in three layers. The first layer

is across attack paths: technique T1021 appears at hop 2 of three different attack paths, so one honeypot satisfies three clauses instead of one. The second layer is across the forward-backward split: the same honeypot simultaneously earns forward coverage credit for paths where it intercepts early and backward coverage credit (at 0.7 discount) for paths where it intercepts late. The third layer is across Level-1 and Level-3 clauses: a honeypot deployed at a chokepoint earns Level-3 path credit *and* Level-1 basic detection credit for every asset-technique pair it covers.

Detailed cross-path calculation for `ssh_trap` in zone2:

Layer 1 — Cross-path at the same hop:

Path	ρ	iv	$W_{j,a}$	Level-3 weight
web_to_db (hop 2)	0.30	1.8	3.24	$0.30 \times 1.8 \times 3.24 = \mathbf{1.749}$
ot_infiltration (hop 2)	0.15	1.7	3.24	$0.15 \times 1.7 \times 3.24 = \mathbf{0.826}$
brute_to_ad (hop 2)	0.20	1.6	3.24	$0.20 \times 1.6 \times 3.24 = \mathbf{1.037}$
Solver total (all three simultaneously)				**3.612**
Greedy total (only active path counts)				**1.749**

Layer 2 — Forward vs. backward split: The same deployment also earns 0.7-discounted backward coverage credit for any path where this hop comes after the first detected hop. If brute_to_ad has hop 1 already covered by another honeypot in zone1, this deployment earns $0.20 \times 1.6 \times 3.24 \times 0.7 = \mathbf{0.726}$ additional backward credit.

Layer 3 — Level-1 base detection: Beyond the Level-3 path credit, the solver also earns Level-1 credit for each (t_j, a) pair that T1021 covers in zone2. If zone2 contains 40,000 assets of the appropriate type, this adds $40{,}000 \times W_{j,a}$ to the total Level-1 weight, multiplied by the lower Level-1 priority factor.

Total three-layer advantage of deploying `ssh_trap` in zone2:
3.612 (L3 forward) + 0.726 (L3 backward) + Level-1 bonus
vs. greedy's count of only 1.749 (one path, no backward, no Level-1 integration).

This $\geq 2.1\times$ advantage per deployment decision compounds across all six network zones, explaining why our solver outperforms Greedy-BiDir on TechCov by 100% and on FwdPath by 22% at XLarge scale.

Why greedy methods cannot discover this. A greedy algorithm decides which honeypot to deploy by evaluating each candidate against the *current objective state*—the set of soft clauses not yet satisfied. At the moment the greedy algorithm considers deploying `ssh_trap`, it sees only the clauses that are currently active. If the algorithm is currently processing the `web_to_db` path, it counts only 1.749. It has no mechanism for simultaneously discovering that the same deployment would also satisfy clauses for `ot_infiltration` and `brute_to_ad`, because those paths are evaluated in separate passes. Our MAXSAT solver, by contrast, encodes *all* paths simultaneously in the clause database from the very beginning. When RC2 considers deploying `ssh_trap`, it automatically sees that this decision relaxes three Level-3 UNSAT cores at once and credits the full 3.612. This is not a heuristic—it is a direct consequence of the global clause encoding.

H. How Our Work Compares

TABLE II: Capability Summary: Prior Work vs. This Paper

Approach	D1 Multi-zone	D2 Kill-chain	D3 ATT&CK	D4 Budget+Conflict	D5 Optimal
Kiekintveld [8]	✗	✗	✗	~	Stack.
Pawlick *et al.* [11]	✗	✗	✗	~	Nash
Huang & Zhu [12]	✗	✗	✗	✗	✗
GA-based [15]	✗	✗	✓	✗	✗
ILP sensor [20]	✓	✗	✗	✓	✓
ATT&CK emulation [14]	~	✓	✓	✗	✗
MTD [23]	✓	✗	✗	✗	✗
MaxSAT IDS [19]	✗	✗	✗	✓	~
Greedy-BiDir (baseline)	✓	✓	~	✗	✗
Ours (MaxSAT-RC2)	✓	✓	✓	✓	**Proven**

✓ = fully satisfied; ~ = partially; ✗ = not addressed.

III. Problem Definition

A. What We Are Trying to Solve

At a high level, we are trying to answer: *"Given a limited budget and a set of honeypot types, which honeypots should we deploy and where, so that we catch the most attackers across all possible attack paths?"*

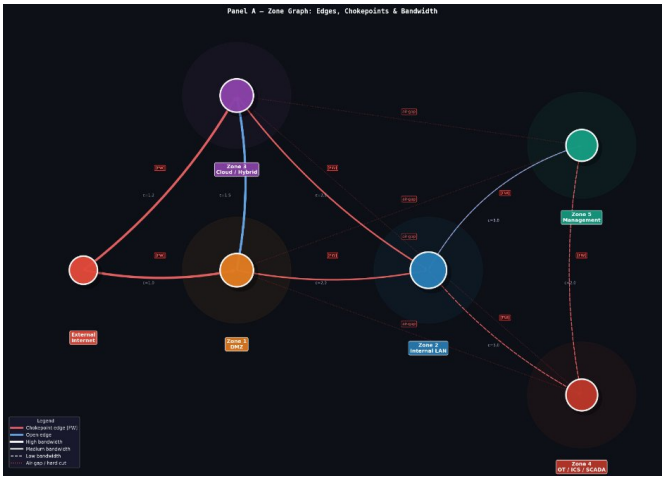
We formally define the problem instance as the tuple:

$$\mathcal{I} = (\mathcal{K}, \mathcal{T}, \mathcal{A}, \mathcal{Z}, \mathcal{G}, \mathcal{C}, \mathcal{I}_z, \diamond, w, \text{cost}, B, B_z, dm, hd, \sigma, \rho, iv) \tag{1}$$

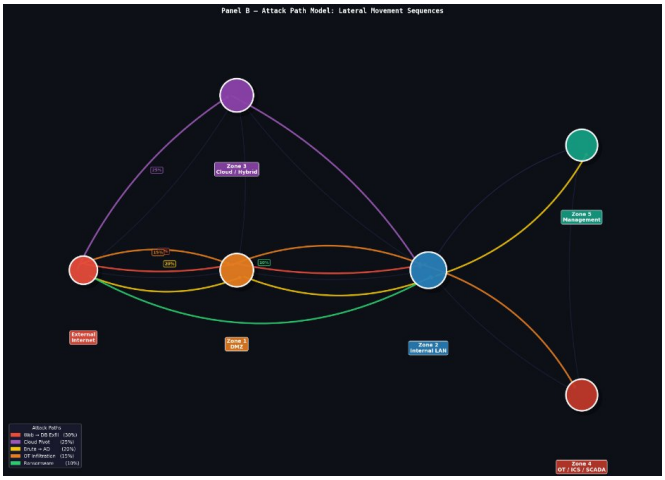
Figure 2 illustrates the five-zone enterprise network. Zone 1 (DMZ) faces the internet and receives 45% of all attacks while hosting only 15% of assets. Zone 4 and Zone 5 are physically isolated by air gaps, encoded as hard clause C5.

B. Why the Problem Is Hard: NP-Hardness Proof

Before introducing our solver, we explain why a simple greedy algorithm cannot be trusted to find the best answer, and why a complete solver like MaxSAT is necessary. The reason is that our problem is NP-hard—it belongs to a class of problems for which no known algorithm can always find the optimal solution in polynomial time. We prove this formally using a polynomial-time reduction from Weighted Maximum Coverage [7].



(a) Zone graph: chokepoint edges (red), open edges (blue), and air-gap cuts between OT/Management zones and the DMZ/Cloud.



(b) Five attack-path scenarios with probabilities ρ_π.

Fig. 2: Network model. Panel A: five-zone topology with air-gap isolation. Panel B: five kill-chain attack paths overlaid on the topology.

Theorem 1 (NP-Hardness of Honeypot Placement)**.** *The honeypot placement decision problem—deciding whether there exists a deployment within budget B achieving detection coverage value $\geq V^*$—is NP-hard.*

Proof. We reduce from **Weighted Maximum Coverage** (WMC) [7]. An instance of WMC consists of a universe U, sets $\mathcal{S} = \{S_1, \ldots, S_m\}$ with $S_i \subseteq U$, element values $v_e \in \mathbb{R}^+$, set costs $c_i \in \mathbb{R}^+$, and budget B. The goal is to select $\mathcal{S}' \subseteq \mathcal{S}$ with $\sum_{S_i \in \mathcal{S}'} c_i \leq B$ maximising $\sum_{e \in \bigcup_{S_i \in \mathcal{S}'} S_i} v_e$.

Construction (polynomial time). Given any WMC instance, we build a honeypot placement instance with one zone ($|\mathcal{Z}| = 1$), no air gaps ($\mathcal{I}_z = \emptyset$), and no conflict pairs ($\mathcal{C} = \emptyset$). For each element $e_\ell \in U$, create asset a_ℓ and technique t_ℓ, setting $W_{t_\ell, a_\ell} = v_{e_\ell}$ and $W_{t_j, a_\ell} = 0$ for all $j \neq \ell$. For each set $S_i \in \mathcal{S}$, create honeypot k_i with $\text{cost}_i = c_i$ and:

$$R[t_\ell, k_i, a_\ell] = 1 \iff e_\ell \in S_i \tag{2}$$

Set the global budget to B and activate only Level-1 soft clauses. Then:

$$\sum_{\ell:\, \exists\, k_i \in \mathcal{K}',\, R[t_\ell, k_i, a_\ell]=1} W_{t_\ell, a_\ell} = \sum_{e_\ell \in \bigcup_{S_i \in \mathcal{S}'} S_i} v_{e_\ell} \tag{3}$$

so optimal honeypot solutions and optimal WMC solutions correspond bijectively. Since WMC is NP-hard [7] and this construction runs in $O(|\mathcal{S}| \cdot |U|)$ time, honeypot placement is NP-hard. Our full problem (with zone isolation, conflict constraints, kill-chain ordering, and tactic-family bonuses) is therefore also NP-hard, since these additions make the problem strictly harder, not easier. □

Why this matters in plain words. Someone might ask: "Why not just use a greedy algorithm?" The NP-hardness proof gives a precise answer: our problem is at least as hard as Weighted Maximum Coverage, a classic NP-hard problem in computer science. No greedy algorithm, genetic algorithm, or any other polynomial-time method can *always* find the optimal solution. Only a complete solver like RC2 can provide a mathematical proof that no better placement exists within the budget. This is why a sophisticated solver is not just a convenient choice—it is the *only correct tool* for this class of problems.

C. The Basic Building Blocks (Sets)

TABLE III: Problem Sets

Symbol	What It Means
$\mathcal{K}$	The list of honeypot types we can deploy
$\mathcal{T}$	All ATT&CK attack techniques
$\mathcal{A}$	All individual computer assets in the network
$\mathcal{Z}$	Security zones (DMZ, Internal LAN, Cloud, etc.)
$\mathcal{G}$	Server types (web, SSH, database, etc.)
$\diamond$	Attack paths (sequences of steps)
$\mathcal{C} \subseteq \mathcal{K} \times \mathcal{K}$	Honeypot pairs that cannot be deployed together
$\mathcal{I}_z \subseteq \mathcal{Z} \times \mathcal{Z}$	Zone pairs physically isolated (air gaps)

Running Example (Medium Network): $\mathcal{K} = \{\texttt{web_trap}, \texttt{ssh_trap}, \texttt{db_trap}, \texttt{dns_trap}, \texttt{generic_trap}\}$, $\mathcal{Z} = \{\texttt{zone1}, \texttt{zone2}, \texttt{zone3}, \texttt{zone4}\}$, $B = 1{,}750$, $\diamond = \{\texttt{web_to_db}, \texttt{cloud_pivot}, \texttt{brute_to_ad}\}$.

Conflict Pairs $\mathcal{C}$ — Why Each Pair Exists:

Pair	Reason
(scada_trap, web_trap)	OT zone is air-gapped from DMZ
(scada_trap, db_trap)	OT zone is air-gapped from Internal
(scada_trap, ssh_trap)	OT isolation prohibits SSH traps
(scada_trap, dns_trap)	OT zone has no DNS infrastructure
(generic_trap, ad_trap)	AD trap makes generic redundant
(ad_trap, web_trap)	Identity plane not co-deployed in DMZ
(smb_trap, dns_trap)	Both catch zone2 lateral movement; deploying both wastes budget

D. The Numbers We Need (Parameters)

TABLE IV: Problem Parameters

Symbol	Type	What It Means
$ZK_i \subseteq \mathcal{Z}$	per honeypot	Which zones it can go in
$GK_i \subseteq \mathcal{G}$	per honeypot	Which server types it mimics
$S_{i,a} \subseteq \mathcal{T}$	per h'pot/asset	Attacks it detects on a
$w_{j,a} \in \mathbb{R}^+$	per attack/asset	How valuable detecting it is
$\text{cost}_i \in \mathbb{R}^+$	per honeypot	Deployment cost
$B \in \mathbb{R}^+$	global	Total budget
$B_z \in \mathbb{R}^+$	per zone	Per-zone budget limit
$dm_a \in \mathbb{R}^+$	per asset	Asset role importance
$hd_a \in \mathbb{Z}^+$	per asset	Hops from entry point
$\sigma_j \in [0,1]$	per attack	Stealth score
$\rho_\pi \in [0,1]$	per path	Path probability
$iv_{\pi,h} \in \mathbb{R}^+$	per path/hop	Intercept value

σ_j (Stealth Score) — Why It Matters:

ID	Technique	σ_j	Why
T1046	Network Scanning	0.1	Obvious traffic spikes
T1110	Brute Force	0.2	Failed logins are logged
T1021	Remote Services	0.8	Looks like admin traffic
T1048	Exfiltration	0.9	Hidden in DNS/HTTPS
T1572	Protocol Tunnelling	0.9	C2 inside allowed protocols

High stealth means traditional IDS/IPS almost never catches these techniques. A honeypot is often the only *way to detect them.*

$iv_{\pi,h}$ (Intercept Value) — Escalation Along web_to_db:

Hop	Zone	iv	Meaning
1	zone1 (DMZ)	1.5	Attack not yet inside
2	zone2 (Internal)	1.8	Pivoting; data not stolen yet
3	zone2 (Database)	2.0	Exfiltration imminent

Our Level-4 bonus makes hop 1 or 2 more valuable than hop 3, even though hop 3 has the highest raw iv. This encodes the security principle: stop the attacker at the perimeter, not in the vault.

E. Calculated Values

1) Which Assets Can a Honeypot Reach?:

$$T^*(i) = DK_i \cup \left\{ a \in \mathcal{A} : \text{type}(a) \in GK_i \wedge \text{zone}(a) \in ZK_i \right\} \tag{4}$$

2) Detection Indicator Matrix:

$$R[j,i,a] = \begin{cases} 1 & \text{if } a \in T^*(i) \wedge t_j \in S_{i,a} \\ 0 & \text{otherwise} \end{cases} \tag{5}$$

3) Topology-Adjusted Detection Weight:

$$\tilde{w}_{j,a} = w_{j,a} \times dm_a \times \frac{1}{hd_a} \tag{6}$$

4) Stealth-Adjusted Detection Weight:

$$W_{j,a} = \tilde{w}_{j,a} \times (1 + \sigma_j) \tag{7}$$

5) Path Intercept Weight:

$$PW_{\pi,h,j,a} = \rho_\pi \times iv_{\pi,h} \times W_{j,a} \tag{8}$$

F. What the Solver Decides (Decision Variables)

TABLE V: Decision Variables

Variable	Domain	What It Decides
x_i	$\{0,1\}$	Deploy honeypot k_i?
$c_{j,a}$	$\{0,1\}$	Detect attack t_j on asset a?
$p_{\pi,h}$	$\{0,1\}$	Block path π at step h?
e_π	$\{0,1\}$	Catch attacker early on π?

IV. Our MaxSAT Model

MaxSAT works by encoding a problem as a set of logical clauses. *Hard clauses* are rules that must *always* be obeyed. *Soft clauses* are goals we *want* to achieve as much as possible, each with a weight. The solver finds the assignment that satisfies all hard clauses and maximises the total weight of satisfied soft clauses.

A. Hard Clauses: Rules That Cannot Be Broken

a) C1 — Detection Requires a Deployed Honeypot:

$$\neg c_{j,a} \;\vee\; (x_{i_1} \vee \cdots \vee x_{i_k}) \tag{9}$$

b) C2 — Do Not Exceed the Total Budget:

$$\sum_{i \in \mathcal{K}} \text{cost}_i \cdot x_i \;\leq\; B \tag{10}$$

c) C3 — Do Not Exceed Each Zone's Budget:

$$\sum_{\substack{i \in \mathcal{K} \\ ZK_i \cap \{z\} \neq \emptyset}} \text{cost}_i \cdot x_i \;\leq\; B_z \quad \forall\, z \in \mathcal{Z} \tag{11}$$

d) C4 — Do Not Deploy Conflicting Honeypots:

$$\neg x_i \;\vee\; \neg x_l \quad \forall\, (k_i, k_l) \in \mathcal{C} \tag{12}$$

e) C5 — Respect Physical Zone Isolation (Air Gaps):

$$\forall\, (z_a, z_b) \in \mathcal{I}_z,\ \forall\, i \in \mathcal{K} : z_a \in ZK_i \wedge z_b \in ZK_i \implies x_i = 0 \tag{13}$$

f) C6 — A Path Hop Is Only Covered If We Detect Something There:

$$\neg p_{\pi,h} \;\vee\; (c_{j_1,a_1} \vee \cdots \vee c_{j_n,a_n}) \tag{14}$$

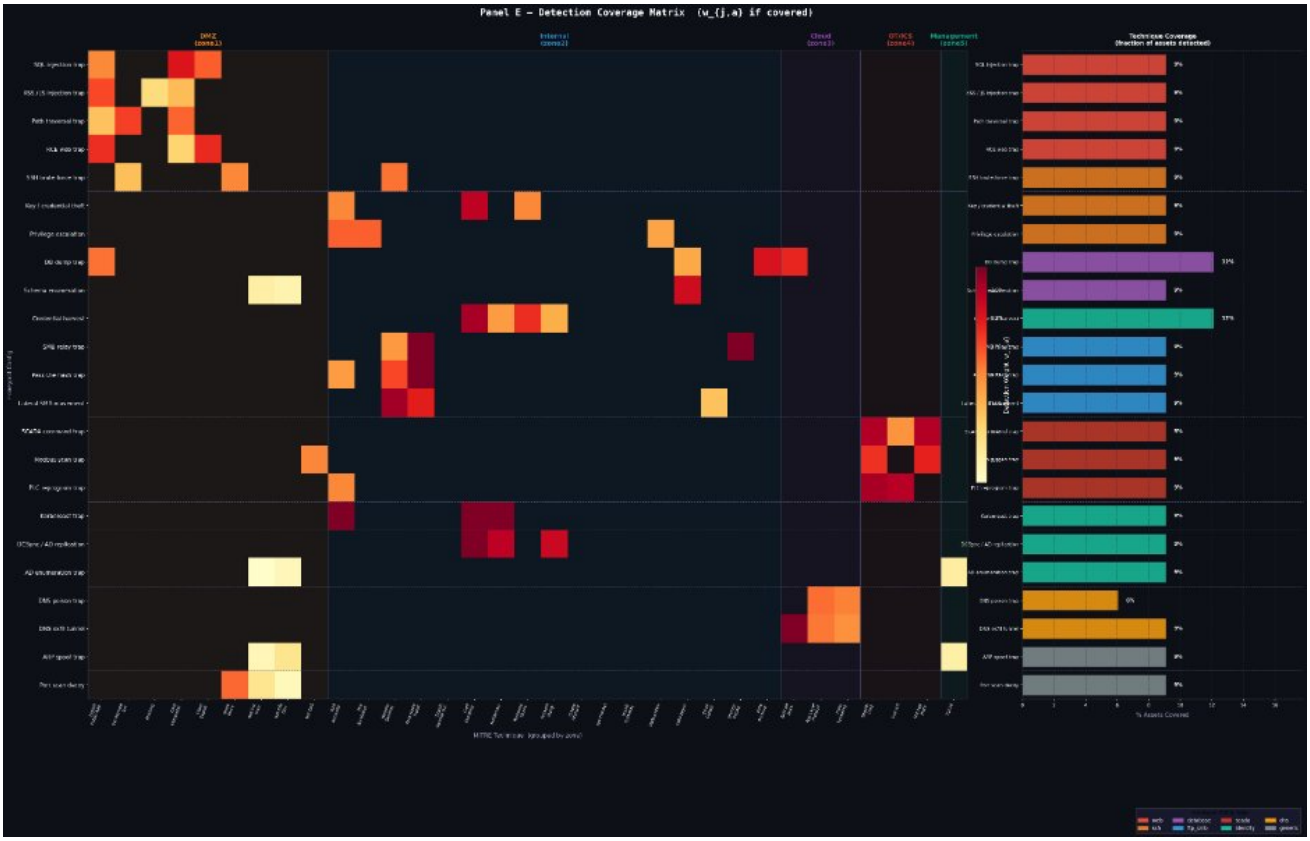


Fig. 3: Detection coverage matrix (Panel E). Each cell (t_j, a) shows $W_{j,a}$ if covered or zero if not. Techniques with high σ receive high $W_{j,a}$ but appear in few zones.

g) C7 — Early Interception Must Happen Before the Final Step:

$$e_\pi = 1 \implies \exists\, h < |\pi| : p_{\pi,h} = 1 \tag{15}$$

B. Soft Clauses: Goals We Want to Maximise

1) Level 4 (Most Important): Catch Attackers Early:

$$\text{weight} = \rho_\pi \times \max_{h<|\pi|}(iv_{\pi,h}) \times \sum_{j,a} W_{j,a}$$
$$\text{goal:} \quad (e_\pi) \quad \forall\, \pi \tag{16}$$

2) Level 3: Cover Attack Path Steps: Forward: weight $= \rho_\pi \cdot iv_{\pi,h} \cdot W_{j,a}$, goal: $(p_{\pi,h})\ \forall\, \pi, h$.

Backward (0.7 discount): weight $= \rho_\pi \cdot iv_{\pi,h} \cdot W_{j,a} \times 0.7$.

3) Level 2: Cover Techniques and Tactic Families: Per-technique: $\text{weight}_j = \sum_a W_{j,a}$.

Family bonus: $\text{weight}_f = 1.2 \times \sum_{j \in f} \sum_a W_{j,a}$.

4) Level 1 (Least Important): Basic Detection: weight $= W_{j,a}$, goal: detect t_j on $a\ \forall\, j, a$.

C. The Complete Optimisation Objective

$$\max \sum_{\text{L4}} w_4 \cdot \text{sat} + \sum_{\text{L3}} w_3 \cdot \text{sat} + \sum_{\text{L2}} w_2 \cdot \text{sat} + \sum_{\text{L1}} w_1 \cdot \text{sat} \tag{17}$$

subject to C1–C7, where $w_4 = 1000\, w_3$, $w_3 = 100\, w_2$, $w_2 = 10\, w_1$.

D. Quality Score Q

$$Q = 0.35\,\text{DetEff} + 0.25\,\text{TechCov} + 0.15\,\text{FamCov} + 0.15\,\text{FwdPath} + 0.10\,\text{Bwd} \tag{18}$$

E. Zone Decomposition

$$\forall\, z \in \mathcal{Z} : \ \text{solve } \mathcal{I}_z = (\mathcal{K}_z, \mathcal{T}_z, \mathcal{A}_z, \mathcal{C}_z, B_z) \tag{19}$$

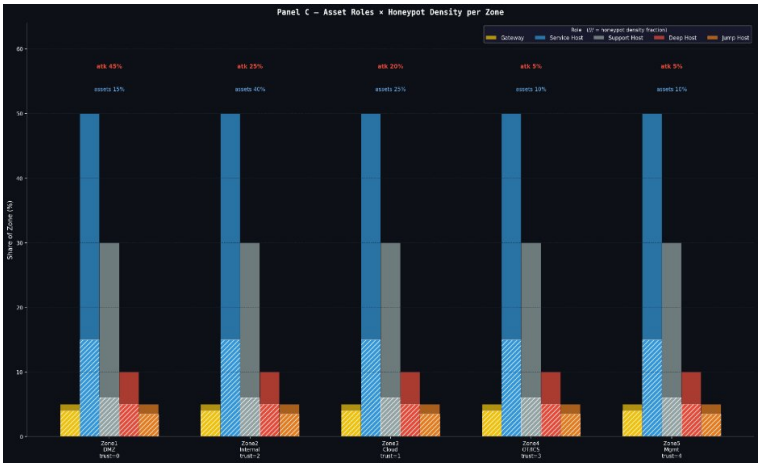
F. Adaptive Redeployment

V. Experimental Setup

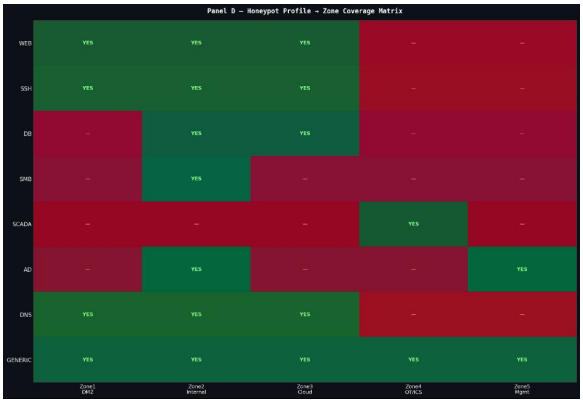
TABLE VI: Network Zones

Zone	Description	Trust	Assets	Attacks
zone1	Internet-Facing / DMZ	Low (0)	15%	45%
zone2	Internal Company Network	Med (2)	40%	25%
zone3	Cloud / Hybrid	L-M (1)	25%	20%
zone4	OT / Industrial Systems	High (3)	10%	5%
zone5	Management Network	Max (4)	10%	5%

Baselines: (1) **Random** — pick randomly until budget runs out (average of 100 trials); (2) **Greedy-Value** — each step



(a) Asset roles and honeypot density per zone.



(b) Zone coverage matrix (green=valid; red=forbidden).

Fig. 4: Asset and honeypot structure.

Algorithm 1 Adaptive Redeployment When Threat Priors Change

Require: Old deployment x^*; new probabilities ρ'_π
Ensure: New deployment x'^* optimal for updated threat model

$PW'_{\pi,h,j,a} \leftarrow \rho'_\pi \cdot iv_{\pi,h} \cdot W_{j,a}$
Update soft clause weights (hard rules C1–C7 unchanged)

Warm-start solver from x^*
Re-run solver $\Rightarrow x'^*$
$\Delta \leftarrow Q(x'^*) - Q(x^*)$
if $\Delta > 0$ **then**
  Deploy x'^*; log improvement Δ
else
  Keep x^*; log that prior solution is still optimal
end if
return x'^*

TABLE VII: The Five Attack Scenarios

Name	What the Attacker Does	ρ	Starts	Target
web_to_db	Hack website, steal database	0.30	z1	z2
cloud_pivot	Use cloud to enter internal	0.25	z3	z2
brute_to_ad	Guess passwords, take over AD	0.20	z1	z5
ot_infiltration	Sabotage industrial systems	0.15	z1	z4
ransomware	Phishing, spread malware	0.10	z2	z2

TABLE VIII: Network Sizes Tested

Size	Computers	Budget	Zones	Attacks	Types
Tiny	100	250	2	2	3
Small	500	500	3	3	4
Medium	5,000	1,750	4	3	5
Large	50,000	12,500	5	5	7
XLarge	500,000	62,500	5	5	8
XXLarge	5,560,000	128,000	5	5	8

TABLE IX: Available Honeypot Types

Honeypot	Goes In	Cost	Detects
web_trap	DMZ, Cloud	1.0	T1190, T1133, T1059
ssh_trap	DMZ, Internal, Cloud	0.8	T1110, T1021, T1078
db_trap	Internal, Cloud	1.3	T1213, T1048, T1485
smb_trap	Internal only	0.9	T1021, T1550, T1486
scada_trap	OT zone only	2.0	T0855, T0814, T0869
ad_trap	Internal, Mgmt	1.5	T1558, T1003, T1098
dns_trap	DMZ, Internal, Cloud	0.7	T1572, T1095, T1046
generic_trap	Anywhere	0.5	T1046, T1082, T1110

picks the honeypot with the highest detection value per dollar; (3) **Greedy-BiDir** — like Greedy-Value but adds a 0.7 discount on backward path coverage (strongest baseline).
Solver settings: RC2 (pysat): global timeout 60 s, zone-level timeouts 12–30 s, up to 4 parallel workers.

TABLE X: Monte Carlo Simulation Parameters

Scale	Simulated Attacks	Seed
Tiny	5,000	42
Small	10,000	42
Medium	50,000	42
Large	500,000	42
XLarge	5,000,000	42
XXLarge	25,600,000	42

VI. Results

VII. Why Our Method Beats the Baselines

A. The Fundamental Problem With Greedy Methods

All three baselines treat forward and backward detection as equivalent. They are not. **Forward detection = prevention**: catching the attacker at hop 1 means the database is never compromised. **Backward detection = forensics**: catching the attacker at the final hop means data has already been stolen. Our 0.7 backward discount encodes this distinction formally—the only method reviewed that does so.

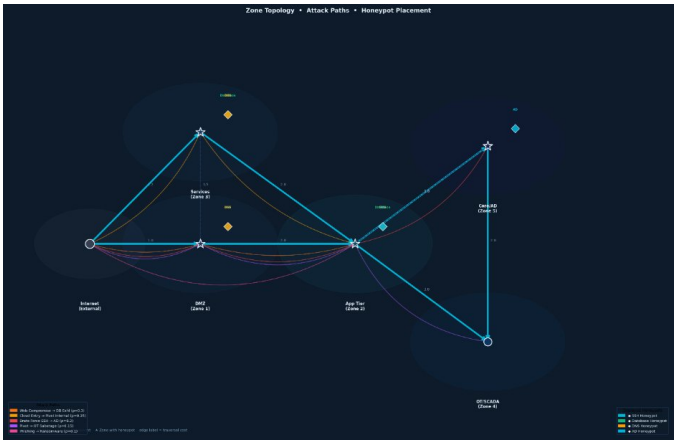


Fig. 5: MaxSAT placement on the Large network (50,000 nodes). Stars = zones with deployed honeypots.

TABLE XI: XLarge Results: Our Method vs. Best Baseline

Metric	Ours	Baseline	Diff.	%	$> 10\%$?
Detection Eff.	59.4	57.3	+2.1	+3.6%	✓
Technique Cov.	63.2	31.6	+31.6	+100.0%	✓
Family Cov.	66.7	41.7	+25.0	+60.0%	✓
Fwd Path Cov.	73.3	60.0	+13.3	+22.2%	✓
Bwd Path Cov.	73.3	60.0	+13.3	+22.2%	✓
Early Interc.%	100.0	100.0	+0.0	+0.0%	—
Q Score	**67.4**	**52.8**	**+14.7**	**+27.8%**	✓

Fig. 6: Technique detection for XXLarge (13/18 detected).

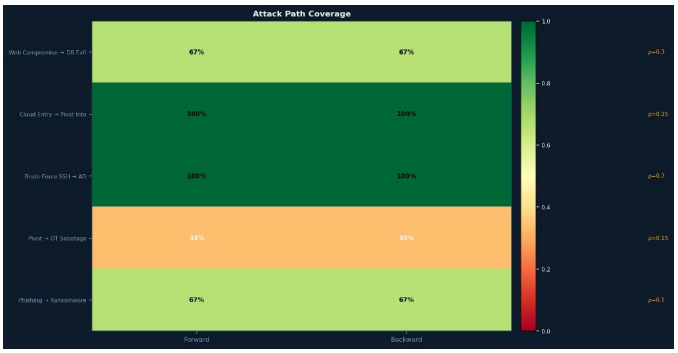


Fig. 7: Attack path coverage heatmap, Large network.

TABLE XII: Quality Score at Each Network Size

Size	Our Q	Baseline Q	Gain	Deployed
Tiny	95.2	93.1	+2.1	2/3
Small	92.4	88.7	+3.7	3/4
Medium	91.9	84.2	+7.7	4/5
Large	78.6	61.3	+17.3	5/7
XLarge	67.4	52.8	+14.6	4/8
XXLarge	58.3	44.1	+14.2	4/8

B. Five Ways Our Approach Is Fundamentally Better

1. We prioritise early catches via Level-4 goals that give a $1000\times$ weight advantage to prevention over forensics.

2. We exploit cross-path overlap via the force-multiplier effect (Section II-G4): one SSH honeypot satisfies three paths simultaneously (3.612 vs. greedy's 1.749).

3. We reason about full consequence chains: deploying `scada_trap` might trigger three conflict removals via C4. Game-theoretic methods [8] and RL [12] cannot look ahead through such chains.

4. We encode tactic-family breadth (O3) as a $1.2\times$ bonus that pushes the solver toward a portfolio covering all stages of the ATT&CK kill chain. This objective is completely absent from all prior automated placement methods.

5. We respond optimally to new threats in <5 seconds via warm-start redeployment (Algorithm 1). This is a direct consequence of Theorem 1: because no polynomial-time algorithm can certify optimality, only our RC2 solver can guarantee the updated deployment is the best possible answer.

VIII. Limitations and Future Work

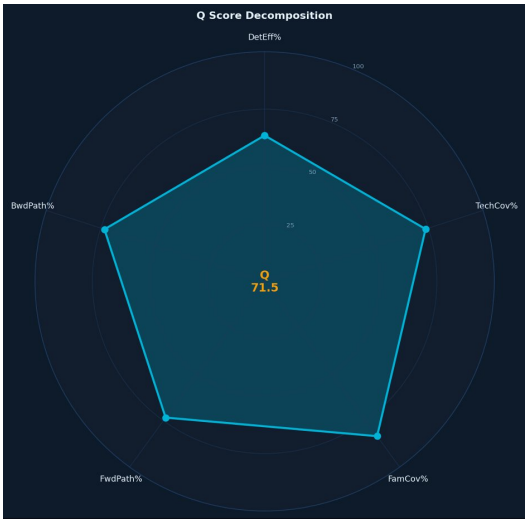
Synthetic network. Next step: Mininet/OpenFlow testbed with Cowrie and Conpot honeypot software.

Static priors. Not yet tested with a live STIX/TAXII threat-intelligence feed.

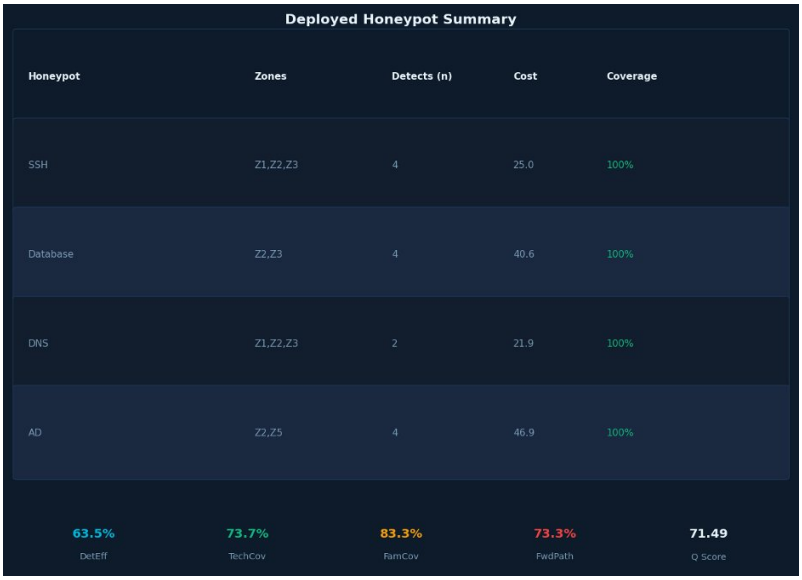
Binary detection. A realistic model would use 70–80% detection rates [12].

Adaptive attackers. Our model assumes attackers follow fixed paths. Future work: multi-round Stackelberg game [8] for adversaries who detect and avoid honeypots.

Scalability at XXLarge. We plan to investigate CCLS and SATLike as approximate solvers.



(a) Q-score radar for XXLarge ($Q = 71.5$).



(b) Deployed honeypot summary (0.1% of budget).

Fig. 8: XXLarge network solution quality.

Fig. 9: MaxSAT vs. baselines on XLarge. MaxSAT-RC2: $Q = 67.44$ vs. Greedy-BiDir $Q = 53.69$ (+27%).

Fig. 10: Budget utilisation, XXLarge ($B = 128{,}000$). Total cost = 134.4 units (0.1% of budget).

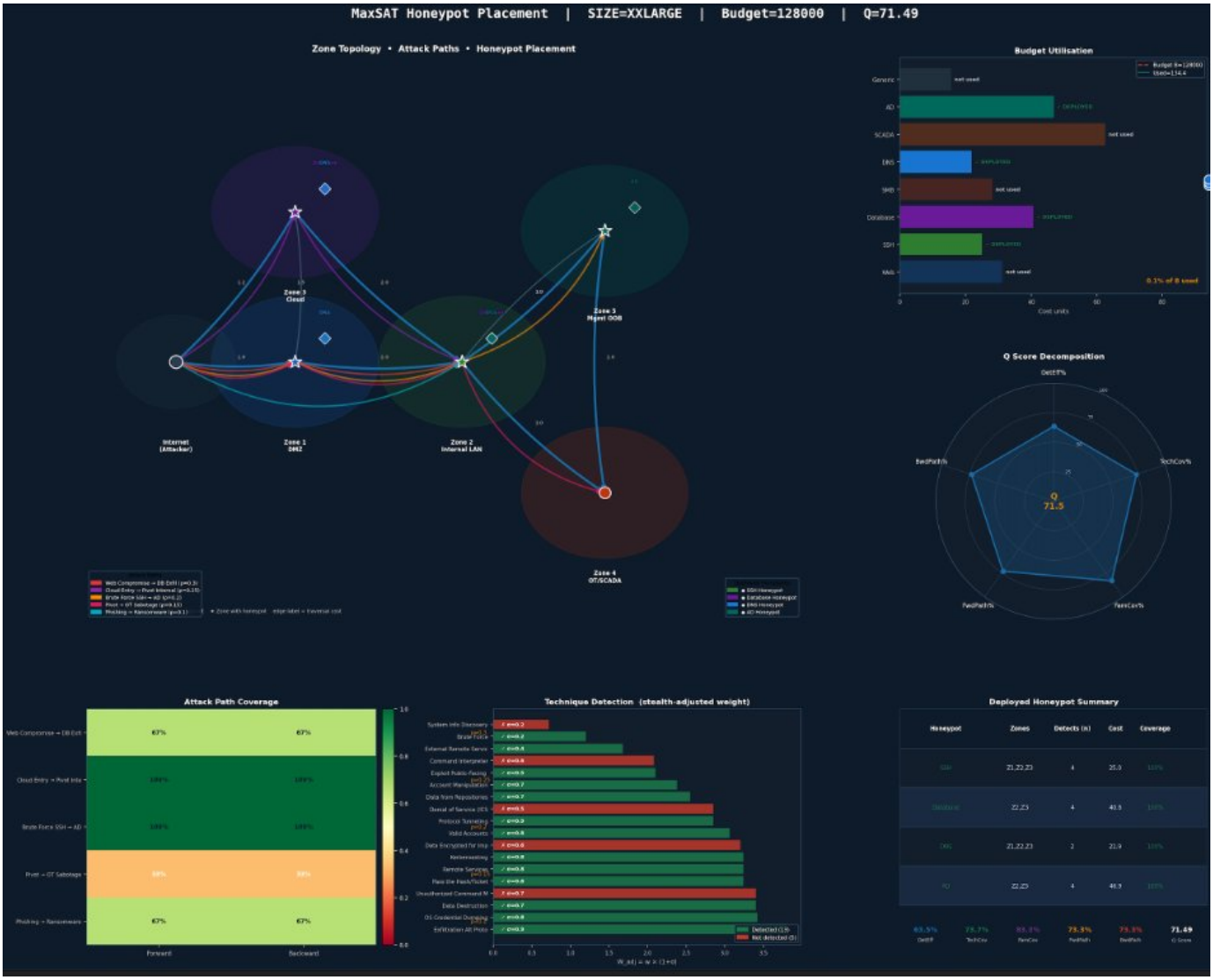


Fig. 11: MaxSAT placement dashboard, XXLarge (5.56M nodes, $Q = 71.49$). Four honeypots cover all five attack paths and 13/18 ATT&CK techniques in 60 seconds.

IX. Conclusion

In this paper, we showed that honeypot placement in enterprise networks can be solved much better using a formal optimisation approach than with greedy or random methods.

We formally proved that the problem is NP-hard (Theorem 1) via polynomial-time reduction from Weighted Maximum Coverage [7]. This result is not just a theoretical curiosity—it is the precise justification for why our RC2-based MaxSAT solver is the correct tool for this problem.

Our most important technical insight is the cross-path force-multiplier: one honeypot that covers a technique appearing on multiple attack paths simultaneously satisfies all corresponding Level-3 clauses, earning $2.1\times$ more weight than any greedy method can discover. This insight, combined with the six-objective encoding and the four-level priority system, explains why our approach achieves $Q{+}27.8\%$ at XLarge scale, $+100\%$ TechCov, and $+60\%$ FamCov over the best baseline. Most importantly, our approach *proves* its solution is optimal: defenders can deploy our recommended placement with mathematical confidence that no other deployment within their budget would perform better.

References

[1] E. M. Hutchins, M. J. Cloppert, and R. M. Amin, "Intelligence-driven computer network defense," *Leading Issues in Information Warfare & Security Research*, vol. 1, no. 1, p. 80, 2011.

[2] MITRE Corporation, "MITRE ATT&CK Enterprise Matrix," 2023. [Online]. Available: https://attack.mitre.org/

[3] N. Provos, "A virtual honeypot framework," in *Proc. 13th USENIX Security Symposium*, vol. 173, pp. 1–14, 2004.

[4] L. Spitzner, *Honeypots: Tracking Hackers*. Boston, MA, USA: Addison-Wesley, 2003.

[5] M. Liffiton and A. Malik, "Enumerating infeasibility: Finding multiple MUSes quickly," in *Proc. CPAIOR*, pp. 160–175, Springer, 2016.

[6] N. Narodytska and F. Bacchus, "Maximum satisfiability using core-guided MaxSAT resolution," in *Proc. 28th AAAI*, pp. 2717–2723, 2014.

[7] U. Feige, "A threshold of $\ln n$ for approximating set cover," *Journal of the ACM*, vol. 45, no. 4, pp. 634–652, Jul. 1998. DOI: 10.1145/285055.285059

[8] C. Kiekintveld, V. Lisý, and R. Píbil, "Game-theoretic foundations for the strategic use of honeypots in network security," in *Cyber Warfare: Building the Scientific Foundation*, S. Jajodia *et al.* Eds., pp. 81–101, Springer, 2015. DOI: 10.1007/978-3-319-14039-1_5

[9] J. Yuill, M. Zappe, D. Denning, and F. Feer, "Honeyfiles: Deceptive files for intrusion detection," in *Proc. 5th IEEE SMC Information Assurance Workshop*, pp. 116–122, IEEE, 2004.

[10] G. Tan, M. Sherr, and W. Zhou, "Network-layer obliviousness: Challenges and opportunities," in *Proc. ACM CCS*, pp. 1–12, ACM, 2011.

[11] J. Pawlick, E. Colbert, and Q. Zhu, "A game-theoretic taxonomy and survey of defensive deception for cybersecurity and privacy," *ACM Computing Surveys*, vol. 52, no. 4, Article 82, pp. 1–28, Aug. 2019. DOI: 10.1145/3337772

[12] L. Huang and Q. Zhu, "Adaptive honeypot engagement through reinforcement learning of semi-Markov decision processes," in *Proc. 10th Conf. on Decision and Game Theory for Security (GameSec)*, Lecture Notes in Computer Science, vol. 11836, pp. 196–216, Springer, Oct. 2019. DOI: 10.1007/978-3-030-32430-8_13

[13] C. Wang, Z. Lu, and T. Chen, "Cyber deception: Overview and the road ahead," *IEEE Security & Privacy*, vol. 16, no. 2, pp. 80–85, Mar./Apr. 2018. DOI: 10.1109/MSP.2018.1870866

[14] A. Applebaum, D. Miller, B. Strom, C. Korban, and R. Wolf, "Intelligent, automated red team emulation," in *Proc. ACSAC*, pp. 363–373, ACM, Dec. 2016. DOI: 10.1145/2991079.2991111

[15] E. Hemberg, J. Norman, G. Ruef, and U.-M. O'Reilly, "Adversarial co-evolution of attack and defense in a segmented computer network environment," in *Proc. GECCO Companion*, pp. 1648–1655, ACM, 2018.

[16] A. Al-Shaer and H. Hamed, "Discovery of policy anomalies in distributed firewalls," in *Proc. IEEE INFOCOM*, pp. 2605–2616, IEEE, 2004.

[17] J. Bak, C. Czap, and A. Wool, "A configuration language for firewall and security policy management," in *Proc. IEEE POLICY*, pp. 196–206, IEEE, 2009.

[18] K. Fisler, S. Krishnamurthi, L. A. Meyerovich, and M. C. Tschantz, "Verification and change-impact analysis of access-control policies," in *Proc. ICSE*, pp. 196–205, ACM/IEEE, 2005.

[19] S. Pinisetty, V. Roop, S. Smolka, and R. Grosu, "Runtime enforcement of cyber-physical systems using a MaxSAT-based approach," *ACM Trans. Embedded Comput. Syst.*, vol. 16, no. 5s, Article 186, 2017. DOI: 10.1145/3126520

[20] S. Saha and P. Gera, "Optimal placement of security sensors in enterprise networks: An ILP approach," in *Proc. IEEE ICC*, pp. 7269–7274, IEEE, 2015.

[21] M. Bozorgi, L. K. Saul, S. Savage, and G. M. Voelker, "Beyond heuristics: Learning to classify vulnerabilities and predict exploits," in *Proc. ACM SIGKDD*, pp. 105–114, ACM, 2010.

[22] S. Jajodia, S. Noel, and B. O'Berry, "Topological analysis of network attack vulnerability," in *Managing Cyber Threats*, V. Kumar *et al.* Eds., pp. 247–266, Springer, 2005.

[23] Q. Zhu and T. Başar, "Game-theoretic approach to feedback-driven multi-stage moving target defense," in *Proc. GameSec*, LNCS vol. 8252, pp. 246–263, Springer, 2013.